

From Experience to Memory-Guided Decision Making

Qixin Xu
2026.6.26

Why Memory?

- Agents interact with the environment over time
- Each episode generates experience: observations, actions, rewards
- Raw experience in context is long, noisy, and hard to reuse for future decisions
- We want the agent to **extract reusable knowledge from experience** and **use knowledge to guide decision making**
- Useful memory should guide future decisions

Memory for Context vs. Memory for Action

Memory for Context

- Used when relevant history is too long
- Stores or retrieves past information
- Main goal: keep information accessible

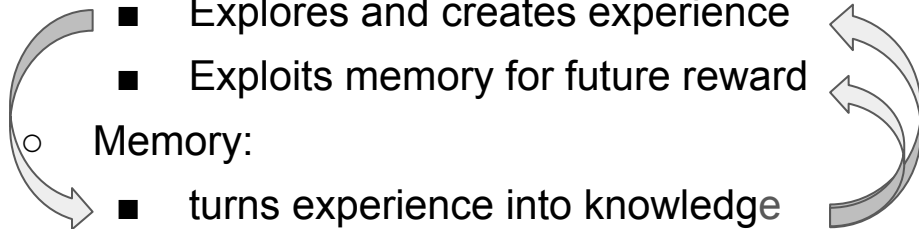
Memory for action

- Used when experience contains reusable knowledge
- Extracts rules, strategies, or world-state beliefs
- Main goal: improve future decisions

Memory for context $\subset$ Memory for action

Where This Fits

- **Prompted reflection / experience reuse:** Reflexion^[2303.11366], ExpeL^[2308.10144]...
- **Context-management memory:** MemGPT^[2310.08560], AgeMem^[2601.01885]...
- **External memory maintenance:** Memory-R1^[2508.19828], Memory-R2^[2605.21768]...
- **Our focus:**
 - Policy using memory:
 - Explores and creates experience
 - Exploits memory for future reward
 - Memory:
 - turns experience into knowledge

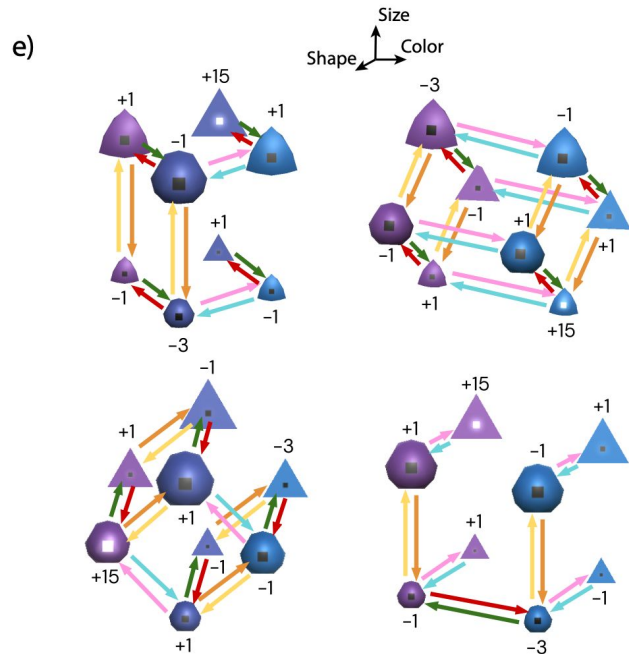
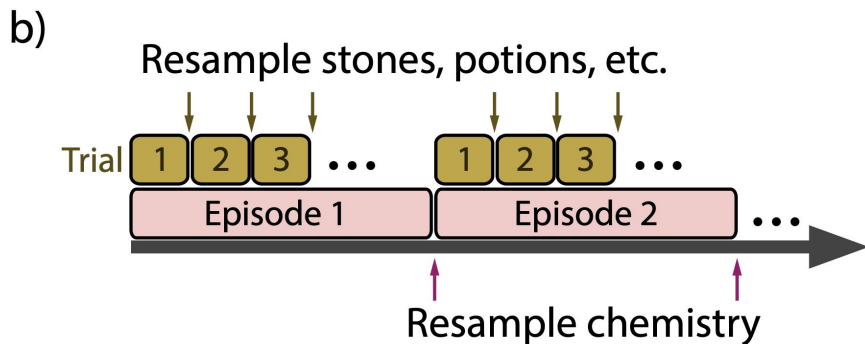


Alchemy: experience $\rightarrow$ knowledge $\rightarrow$ action loop

- Each **episode** has hidden chemistry rules and several **trials**
- **Trials** within same episode share some chemistry rule
- Each **trial** has several **turns** for agent to interact with the environment



- The agent must discover rules through interaction
- Past trials contain useful experience
- Future trials require reusing what was learned



What Should Memory Capture?

- A trial gives the agent local experience:
 - which potion was tried
 - which stone it was applied to
 - how the stone changed
 - whether reward improved
- Useful memory should abstract this into reusable knowledge:
 - potion effects
 - high-value transformations
 - bad / no-op transformations
 - recipes for producing valuable stones

"Yellow potion changed blue small round +1 into blue large round +15"

"Yellow can increase value for blue round stones; try it before cashing out."

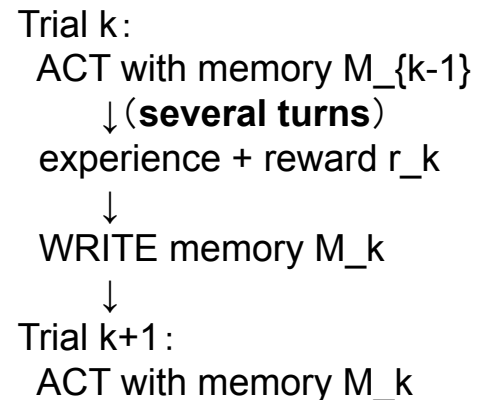
Baselines

- Performance: raw reward / oracle reward
- Improvement: $\text{avg}(\text{last 5 trials}) - \text{avg}(\text{first 5 trials})$

Models	Memory	Performance	Improvement	Analysis
Qwen3-4B	-	0.260 $\pm$ 0.028	0.070 $\pm$ 0.040	Fail to explore, not enough useful information feeded into memory
	✓	0.240 $\pm$ 0.038	0.021 $\pm$ 0.064	
Qwen3.5-4B	-	0.270 $\pm$ 0.050	0.121 $\pm$ 0.056	Model explore decently but model cannot simply do in-context learning
	✓	0.319 $\pm$ 0.062	0.098 $\pm$ 0.055	
Claude 4.8 opus	-	0.674 $\pm$ 0.034	0.142 $\pm$ 0.055	Strong model do in-context learning well, memory will loss information
	✓	0.618 $\pm$ 0.042	0.068 $\pm$ 0.050	

ACT / WRITE Loop

- The agent alternates between two behaviors:
 - **ACT**: use current memory to interact with the environment
 - **WRITE**: update memory from the latest trial experience
- Memory written after one trial is used in later trials
- Both behaviors matter:
 - bad ACT produces uninformative experience
 - bad WRITE loses useful knowledge
 - bad memory use leads to poor future actions



How Do We Reward Memory Writing?

- There is no direct label for the “correct” memory, but ideal memory are at least:
 - Factually correct
 - Benefit downstream performance
- Several proxy signals:

1. **Transition prediction**: Does memory predict future observed transitions?

$$\text{reward}(M_k) = \text{mean}_{(s,a,s') \in F_k} 1 [\text{model predicts } s' \text{ from } (M_k, s, a)]$$

2. **Downstream score**: How does the next trial score performs?

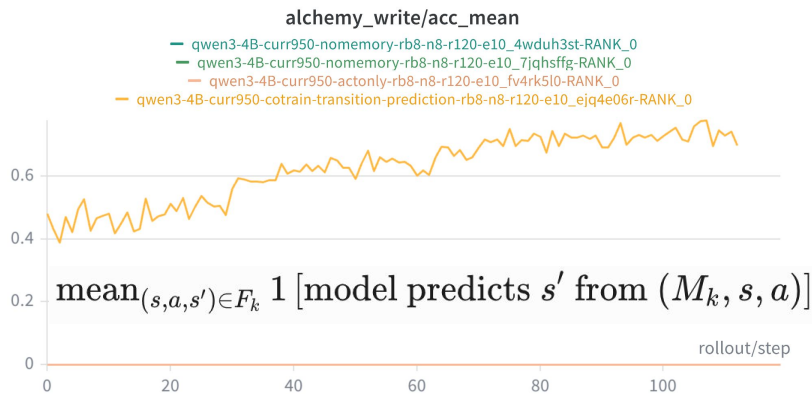
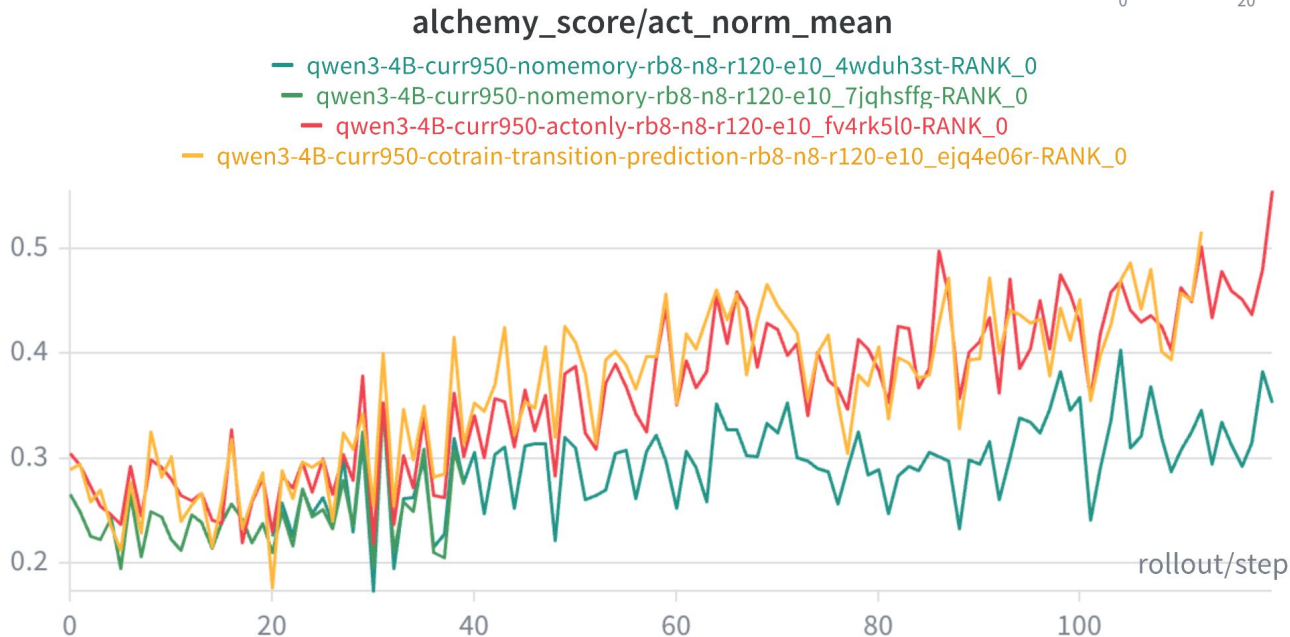
$$\text{reward}(M_k) = r_{\{k+1\}}$$

3. **Improvement score**: Does the next trial score improve?

$$\text{reward}(M_k) = r_{\{k+1\}} - r_k$$

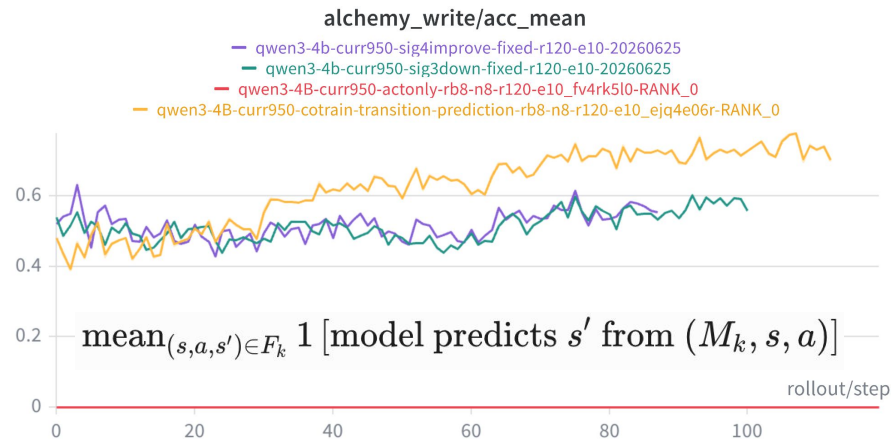
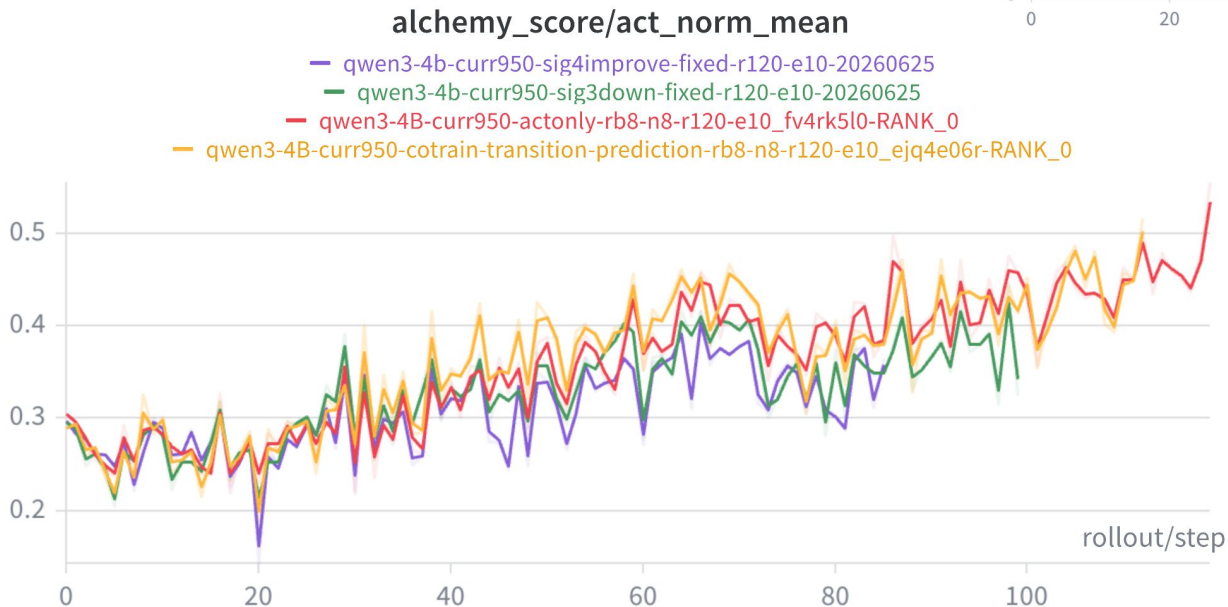
Initial results - baselines

- No memory
- WRITE not trained (Only train ACT)
- Train WRITE using **Transition prediction**



Initial results - WRITE reward

- WRITE not trained (Only train ACT)
- Train WRITE using **Transition prediction**
- Train WRITE using **Downstream score**
- Train WRITE using **Improvement score**



Failure modes

Transition prediction signal: Predicting transition well $\neq$ good memory

Potion Effects

Pink

- Acts on blue large round stones (color and shape), transforming them into blue small round stones (shape changes from large to small) and increasing reward from -1 to +1 (positive change).
- Acts on blue large pointy stones (color and shape), transforming them into blue small pointy stones (shape changes from large to pointy $\rightarrow$ small pointy) and increasing reward dramatically from +1 to +15 (major positive change).
- No effect on purple stones (large round or pointy), or on repeated exposure (no further improvement).
- No effect on other stone types (e.g., purple large pointy or purple large round).

Red

- No observed effect on any stone tested (blue, purple large round, purple large pointy).
- No change in reward or shape.

...

Memory at step 99: better
mapping table

Next:

1. Which reward for WRITE
2. How do we reward ACT?

We want ACT:

- Exploit memory to get higher rewards from the environment
- Explore environment extensively to gain more information for memory

$$\Delta(\mathcal{H}(M_t) - \mathcal{H}(M_{t-1}))$$

3. More real-world environment?

- Learn rules(gain prior) on the fly
- Understanding rules > infer from raw trajectories
- Continue learning, not static i.i.d QA